



Auto-accel: a SW/HW co-design framework with adaptive unified buffer mapping and power-of-two quantization for FPGA-based object detection accelerators

Junha Ko¹ · Dongjun Lee¹ · Youngchan Kim¹ · Hyun Kim¹

Received: 9 July 2025 / Accepted: 24 October 2025

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract

Recent advancements in convolutional neural network (CNN)-based object detection models have significantly improved detection accuracy; however, these improvements have come at the cost of increased computational and memory demands, posing challenges for efficient deployment in resource-constrained edge environments. To address these challenges, existing CNN accelerators have primarily focused on enhancing computational efficiency through the adoption of lightweight models such as MobileNetV1. Nevertheless, systematic analyses of layer-wise memory requirements have been relatively lacking, often resulting in inefficient utilization of on-chip memory (OCM) resources during deployment. To overcome these limitations, this paper proposes *Auto-Accel*, a SW/HW co-design solution for CNN accelerators that simultaneously enables energy-efficient computation and improves memory resource utilization. On the SW side, *Auto-Accel* effectively reduces hardware resource utilization by proposing a fused quantization technique based on the sum-of-power-of-two scaling factor approximation. On the HW side, *Auto-Accel* includes an adaptive unified buffer mapping that efficiently reallocates buffer resources according to the layer-wise memory requirements of activations and weights. Furthermore, to maximize data reuse and reduce off-chip memory accesses, we propose a tile-based adaptive pipelined dataflow, which maximizes computational and energy efficiency. The MobileNetV1-SSD lite accelerator equipped with the proposed *Auto-Accel* achieves an energy efficiency of 11.5 FPS/W when implemented on a ZCU102 board, representing an improvement of approximately 1.33× to 4.41× in energy efficiency compared to prior studies.

Keywords Convolutional neural network · Hardware accelerator · Field-programmable gate array (FPGA) · Design optimization · Dataflow

1 Introduction

With the growing importance of low-latency processing and privacy preservation, on-device AI—which enables computation without transmitting data to cloud servers—has been attracting significant attention, and its application in the field of computer vision (CV), where real-time processing is crucial, is highly anticipated [34]. Among these tasks, object detection, a representative CV application, has been deployed across various industrial domains, including autonomous driving, smart factories, and video surveillance, owing to its robust object detection capabilities [6, 44, 45]. In particular, the advancement of convolutional neural networks (CNNs) has significantly improved the accuracy of object detection models; however, the development trend of increasing model size and depth has introduced many challenges in implementing these models on

Junha Ko and Dongjun Lee have contributed equally to this work.

✉ Hyun Kim
hyunkim@seoultech.ac.kr

Junha Ko
junha.ko@seoultech.ac.kr

Dongjun Lee
dj.lee@seoultech.ac.kr

Youngchan Kim
yckim9978@seoultech.ac.kr

¹ Department of Electrical and Information Engineering,
Research Center for Electrical and Information Technology,
Soul National University of Science and Technology, 232
Gongneung-ro, Nowon-gu, Seoul 01811, Korea

resource-constrained hardware (HW) [5, 15]. To address these challenges, various model compression techniques have been proposed to reduce computational complexity and memory usage while minimizing accuracy degradation. Representative approaches include lightweight model architectures [14, 33], quantization [4, 16, 18], and pruning [17, 20], all of which have garnered considerable attention. In addition, image pre-processing techniques that reduce input resolution have been employed in conjunction with these approaches to further alleviate computational burdens [32, 35].

In addition to model compression techniques, continued research has focused on developing accelerators that implement object detection models on HW platforms, such as field-programmable gate arrays (FPGAs) and application-specific integrated circuits (ASICs), which offer low power consumption and excellent parallel processing capabilities, thereby addressing the severe power consumption issues associated with graphic processing units (GPUs) [1, 9, 19]. However, when implementing these models on HW platforms (i.e., FPGA and ASIC), insufficient consideration of the efficiency of deployed compute units and memory resources can lead to frequent off-chip memory accesses, thereby resulting in increased power consumption [43].

To comprehensively address these issues, this paper proposes *Auto-Accel*, a software (SW)/HW co-design solution optimized for designing power-efficient FPGA-based object detection accelerators. Within *Auto-Accel*, the SW component facilitates the deployment of large CNN-based object detection models in an HW-friendly manner, while the HW component significantly reduces on-chip memory (OCM) usage and eliminates frequent off-chip memory accesses. Through this SW/HW co-design solution, CNN-based object detection models can be implemented on FPGA platforms in an energy-efficient manner. The main contributions of this paper are as follows:

- On the SW side, we propose a fused quantization method based on sum-of-power-of-two (PoT) scaling factor approximation (SPoT-SA). By designing a shift-add fused quantization module, the proposed approach achieves a 10.36× reduction in digital signal processor (DSP) resource usage compared to previous studies.
- On the HW side, we propose adaptive unified buffer mapping (AUBM) and tile-based adaptive pipelined dataflow (TAP) to efficiently utilize HW resources while minimizing off-chip memory accesses. Through these techniques, the proposed approach reduces BRAM usage by 57.94% and achieves a 2.13× improvement in throughput compared to previous studies.
- The MobileNetV1-SSD lite accelerator equipped with the proposed *Auto-Accel* demonstrates 11.5 FPS/W and 1.194 FPS/DSP, demonstrating that the proposed HW

architecture effectively improves both computational and energy efficiency.

2 Background and motivation

2.1 MobileNetV1-SSD lite

2.1.1 Architectural overview and computational optimization

A key characteristic of the MobileNetV1 [14] network is its use of depth-wise separable convolution (DSC), which significantly reduces the computational cost and the number of parameters compared to conventional CNNs [11, 25]. The computational cost of the standard convolution (SC) commonly used in CNNs, denoted as C_{SC} , can be expressed by the following equation:

$$C_{SC} = D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F, \quad (1)$$

where $D_K \cdot D_K$ denotes the kernel size, M is the number of input channels (ICs), N is the number of output channels (OCs), and $D_F \cdot D_F$ represents the size of the feature map. As shown in Eq. (1), the computational cost of SC is determined by the product of the ICs, OCs, kernel size, and feature map size. In contrast, DSC reduces the computational cost by decomposing the SC operation into two stages: a depth-wise convolution (DWC) and a point-wise convolution (PWC). The DWC performs grouped convolution operations for each IC, while the PWC performs a weighted summation across channels. The computational cost of DSC, denoted as C_{DSC} , can be expressed as follows:

$$C_{DSC} = D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F. \quad (2)$$

Based on Eqs. (1) and (2), the computation reduction ratio of DSC compared to SC, denoted as C_{DSC}/C_{SC} , can be expressed as follows:

$$\frac{C_{DSC}}{C_{SC}} = \frac{1}{N} + \frac{1}{D_K^2}. \quad (3)$$

In Eq. (3), assuming a typical CNN configuration with $D_K = 3$ and $N \gg 1$, the ratio $C_{DSC}/C_{SC} \approx 1/9$ indicates that the computational cost of DSC is reduced by approximately 8–9× compared to SC. Leveraging this computational efficiency, MobileNet has been widely adopted as a backbone network for various models [3, 29, 39].

The single-shot multibox detector (SSD) [26] is a representative one-stage detection model that simultaneously predicts bounding boxes and classification scores from multi-scale feature maps. MobileNetV1-SSD lite adopts the previously described MobileNetV1 as its backbone

network and further replaces the SC operations in SSD with DSC, resulting in a lightweight model that is well-suited for deployment in edge environments compared to other CNN-based object detection models.

2.1.2 Memory characteristic for efficient deployment

As described in Sect. 2.1.1, MobileNetV1-SSD lite effectively reduces the substantial computational cost inherent in conventional CNN-based object detection models by leveraging the structural advantages of DSC. Nevertheless, the overall memory requirement remains excessive for resource-constrained HW implementation, and accelerator designs that do not consider these memory requirements can induce frequent off-chip memory accesses, resulting in high power consumption [2, 43].

Accordingly, to enable the energy-efficient HW deployment of MobileNetV1-SSD lite, we analyze the memory requirements of the model across different layer intervals. Figure 1 presents the layer-wise memory requirements for activations and PWC weights in MobileNetV1, which serves as the backbone of MobileNetV1-SSD lite. Since PWC accounts for 97.74% of the weight memory in MobileNetV1-SSDlite, this paper focuses on comparing the memory requirements of PWC weights and activations. As shown in Fig. 1, in the early stages of the network, the large feature map sizes lead to relatively high activation memory requirements. In contrast, in the later stages of the network, as the number of channels increases, the memory requirements of PWC weights become dominant. For instance, in layer 1, the activation requires approximately 1406 KB of memory, while the PWC weights require only 2 KB, whereas in layer 13, the activation requires 100 KB, while the PWC weights require 1024 KB of memory. In other words, MobileNetV1-SSD lite exhibits an asymmetric memory usage pattern, where the primary contributor to memory requirements shifts from activations to weights as the network depth increases.

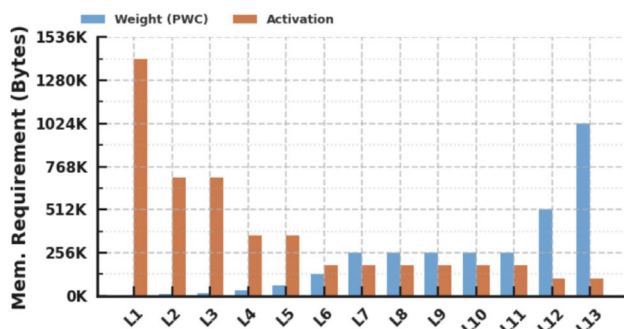


Fig. 1 Memory footprint of weights and activations across MobileNetV1 layers

If HW is designed without considering these memory requirements, the utilization of buffers may degrade regardless of the actual memory needs of each layer, leading to inefficient use of OCM resources. Although configuring the sizes of the activation and weight buffers based on their respective peak memory values allows full utilization of memory resources during the computation of specific layers, this approach typically results in excessive resource allocation relative to the actual requirements for most layers, leading to persistently low memory utilization. Therefore, efficient CNN deployment on edge devices requires a memory strategy that maximizes OCM utilization by adapting to each layer's memory needs.

2.2 Pre-processing and model compression methods

2.2.1 Resize network

In object detection models, input resolution has a significant impact on accuracy, and in general, higher resolution improves performance. However, this also rapidly increases computational and memory demands, leading to excessive overhead in edge environments [10]. To address this issue, previous studies have proposed learnable resizer network architectures that can effectively downscale input images while minimizing accuracy degradation [32]. The resizer network consists of two parallel paths: one path reduces the resolution using a bilinear down-sampling module, while the other preserves feature information through a shallow CNN block. The outputs of these two paths are integrated via averaging, and the entire structure can be trained in an end-to-end manner. By adding this dual-path structure before CNN-based models, it becomes possible to effectively reduce the activation memory requirements by lowering the resolution while maintaining performance.

2.2.2 Quantization

Quantization is a representative model compression technique that reduces the computational and memory requirements of deep learning models by converting floating-point data into low-bit fixed-point or integer representations. It is widely utilized to achieve high area and energy efficiency, particularly in edge environments. In general, uniform quantization applies the following quantization operation to an input x :

$$x_q = \text{clip}\left(\left\lfloor \frac{x}{s} \right\rfloor, -2^{b-1}, 2^{b-1} - 1\right), \quad s = \frac{R_{\max}}{2^{b-1} - 1} \quad (4)$$

where $R_{\max}$ denotes the maximum value of x and s is the scaling factor, and $\lfloor \cdot \rfloor$ represents round-to-nearest. Through Eq. (4), the real-valued input x and weight w are mapped to integer values x_q and w_q with a bitwidth of b . After performing the convolution operation* between the quantized

input x_q and weight w_q , the result is dequantized through the two scaling factors and mapped back to the real-value a_{dq} as follows:

$$a_{dq} = s_x s_w (x_q * w_q). \quad (5)$$

In integer quantization, floating-point multipliers are replaced with integer multipliers, offering advantages such as improved execution speed on HW and more than a two-fold reduction in the area of arithmetic units [7], among other benefits [30].

2.2.3 Fused quantization

The dequantization–quantization process between consecutive layers involves multiplying by $s_x s_w$ for dequantization as shown in Eq. (5), followed by scaling and projection using s as in Eq. (4). In this process, the use of scaling factors requires wide-bitwidth multipliers and dividers for floating-point operations, resulting in a large arithmetic unit area and increased latency, which leads to HW inefficiencies. To address this issue, fused quantization is employed, where the scaling factors used in the dequantization–quantization processes between consecutive layers are precomputed offline and applied in an integrated manner. This process can be expressed as follows:

$$\begin{aligned} a_q &= \text{clip}\left(\left\lfloor \frac{a}{s_a} \right\rfloor, -2^{b-1}, 2^{b-1} - 1\right) \\ &= \text{clip}\left(\left\lfloor \frac{s_x s_w (x_q * w_q)}{s_a} \right\rfloor, -2^{b-1}, 2^{b-1} - 1\right). \end{aligned} \quad (6)$$

Here, s_x and s_w denote the scaling factors for the input and weight of the previous layer, respectively, while s_a is the scaling factor used for quantizing the input values of the subsequent layer. When these scaling factors are precomputed offline, the fused quantization equation incorporating the fused scaling factor s_f can be expressed as follows:

$$a_q = \text{clip}\left(\left\lfloor s_f (x_q * w_q) \right\rfloor, -2^{b-1}, 2^{b-1} - 1\right), \quad s_f = \frac{s_x s_w}{s_a}. \quad (7)$$

Figure 2 illustrates the schematic flow and differences between conventional quantization and fused quantization. By employing fused quantization, the dequantization operation of the previous layer and the quantization operation of the current layer are combined into a single scaling factor, effectively eliminating unnecessary intermediate operations and reducing overall latency and computational overhead. However, since the fused scaling factor s_f is typically represented as a floating-point value, directly applying it in HW requires a floating-point multiplier. Compared to integer-based arithmetic units, floating-point multipliers consume approximately 25 times more look-up table (LUT) resources

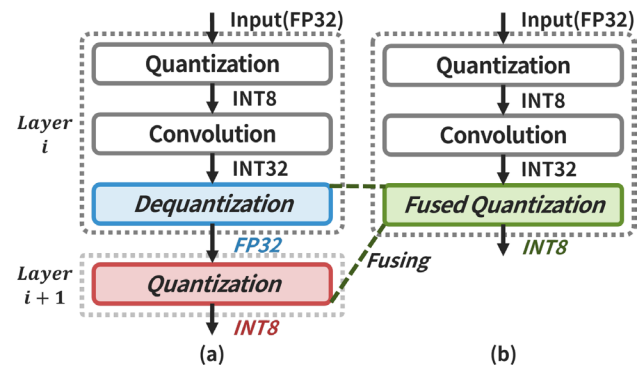


Fig. 2 Comparison of **a** standard quantization with separate dequantization and re-quantization, and **b** fused quantization

and require three times more DSP resources [12]. Therefore, for deployment on resource-constrained HW, methods that reduce or approximate floating-point operations while preserving accuracy are essential.

2.3 Limitations of existing SSD accelerators

Edge devices are constrained by limited computational resources and power budgets, making efficient CNN deployment a challenge of both performance and hardware resource balance. The key lies in enhancing structural efficiency to maximize performance within tight constraints. To meet these demands, prior work on MobileNet-based CNN accelerators has explored HW-efficient architectures using various computation optimization techniques. These include designing dedicated compute units for specific operations (*e.g.*, SC, DWC, PWC) or optimizing general-purpose engines based on the characteristics of each computation pattern.

Liu et al. [27] proposed a co-design approach that preprocesses parameters (*e.g.*, layer configurations, weight quantization, and scaling factors) in the processing system (PS) before transferring them to the programmable logic (PL). However, their use of a single compute unit led to low DSP utilization for PWC operations and limited efficiency for DSC. Hong et al. [13] introduced dedicated compute units for DSC and a tiling technique, achieving high throughput. Yet, this came at the cost of excessive hardware usage (*e.g.*, LUTs, FFs, BRAMs). Their design also allocates fixed-size buffers for activations and weights, which—as discussed in Sect. 2.1.2—cannot adapt to layer-wise memory variations in MobileNetV1. This static buffer structure reduces OCM utilization in certain layers, degrading overall memory efficiency. Furthermore, most existing accelerators focus on computation parallelization and throughput, but lack structural flexibility and efficient resource usage. To overcome these limitations, it is essential to design compute units tailored to the target model and adopt a dynamic memory structure that maximizes limited OCM utilization. Flexible

memory management and adaptive dataflow that account for layer-wise memory needs can further enhance both memory and compute unit utilization.

3 Proposed software solution in AUTO-Accel

3.1 Sum of PoT scaling factor approximation

PoT approximation has been used as an HW-friendly approximation method by replacing floating-point multiplications with shift operations, thereby reducing computation and resource usage in HW. By leveraging this, the floating-point multiplication in Eq. (7) for fused quantization can be replaced with bit-shift operations, reducing the HW resource consumption associated with floating-point multipliers. When the fused scaling factor s_f is approximated using PoT and expressed in the form of s_p , it can be represented as follows:

$$\begin{aligned} s_f &\approx s_p = 2^{\lfloor \log_2 s_f \rfloor} \\ a_q &\approx \text{clip}(\lfloor s_p(x_q * w_q) \rfloor, -2^{b-1}, 2^{b-1} - 1) \\ &= \text{clip}(\lfloor (\lfloor \log_2 s_f \rfloor \gg (x_q * w_q)) \rfloor, -2^{b-1}, 2^{b-1} - 1). \end{aligned} \quad (8)$$

Through Eq. (8), it becomes possible to eliminate floating-point operations in the implementation of fused quantization, enabling the realization of a shift-based quantizer. However, when applying PoT approximation to scaling factors that are sensitive to accuracy, the range of values that can be accurately approximated becomes limited, leading to significant approximation errors, which can, in turn, result in substantial accuracy degradation.

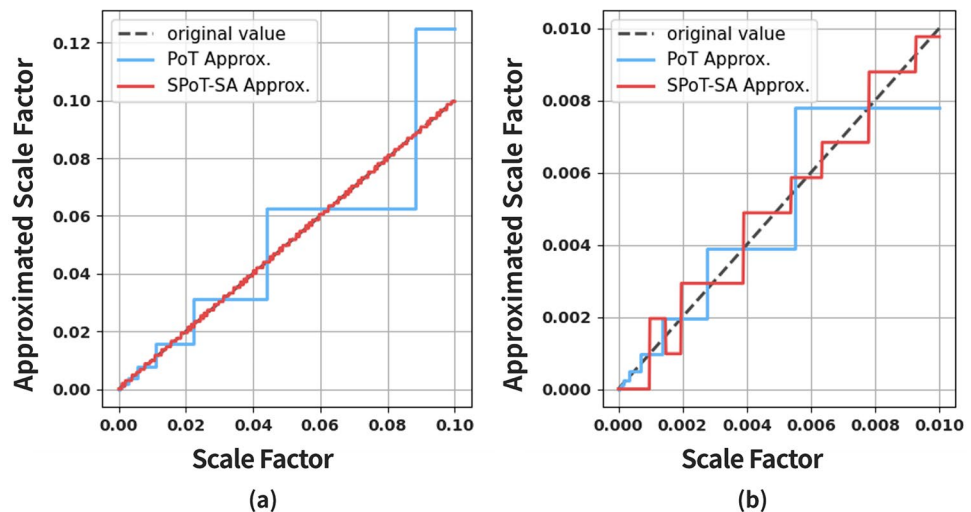
To address this issue, this study proposes the SPoT-SA technique, which extends the approach of the existing Additive Power-of-Two (APoT) [23] method to approximate the scaling factor in fused quantization as a sum of multiple PoT terms. SPoT-SA achieves a much finer scaling factor approximation than single PoT methods and can be implemented with shift-add operations alone, preserving hardware efficiency with minimal accuracy loss. Figure 3 compares the original scaling factor with the approximation results of the single PoT approximation and the proposed SPoT-SA. The single PoT approximation (blue) can only perform a coarse, staircase-shaped approximation, resulting in a significant difference from the original value, whereas SPoT-SA (red) approximates the original scaling factor more precisely, reducing approximation errors and maintaining accuracy. The scaling factor approximated through SPoT-SA, denoted as s_{ap} , is defined as follows:

$$s_f \approx s_{ap} = \sum_{i=1}^T 2^{-k_i}, \quad T \leq B, \quad k_i \in \{1, 2, \dots, k\}. \quad (9)$$

Here, B denotes the maximum number of PoT terms to be used, and k represents the maximum selectable exponent value. The s_{ap} in Eq. (9) shares the same structure as the fixed-point representation. However, by appropriately constraining the parameters B and k to balance the trade-off between numerical precision and hardware cost, the overall hardware efficiency can be substantially enhanced.

We utilize a greedy algorithm to determine the optimal combination of PoT terms, which is fully executed offline to eliminate any additional runtime overhead. At each iteration, the term that minimizes the difference with the current residual is selected from a predefined set of PoT candidates,

Fig. 3 Comparison of scaling factor approximation using single PoT(blue) and SPoT-SA(red)



$2^{-1}, 2^{-2}, \dots, 2^{-k}$. The selected term is accumulated into the final approximated value s_{ap} , and the residual is updated by subtracting the selected term. This process repeats until a maximum of B terms are selected or the residual becomes smaller than a threshold ϵ (set as $\epsilon = 10^{-6}$ in this work). By iteratively selecting the PoT term closest to the residual, the scaling factor is approximated as the sum of multiple 2^{-k_i} terms. Each term can be represented using bitwise shifts without multiplications, and the overall approximated value can be computed using only shift-add operations.

$$s_f(x_q * w_q) \approx \sum_{i=1}^T ((x_q * w_q) \gg k_i), \quad (10)$$

In hardware implementation, the floating-point multiplication between the activation and scaling factor is replaced by a shift-add operation, as shown in Eq. (10), to improve hardware efficiency.

To maximize computational efficiency, we designed HW modules capable of processing shift-add-based PoT combinations. Figure 4 illustrates the hardware operation of the proposed SPoT-SA-based quantization module, composed of a four-stage pipeline. In the Shift stage, shift operations are performed based on the precomputed PoT terms (k_i), enabling multiplication-free scaling by 2^{-k_i} . In the Add stage, all shifted activations are accumulated and forwarded to the Rounding stage in a fixed-point format. In the Rounding stage, the fractional component is utilized to perform rounding, after which the rounded integer data are forwarded to the Clipping stage. In the Clipping stage, overflow logic is applied to constrain the output values within the target quantization bitwidth. The proposed shift-add approximation effectively eliminates wide bitwidth fixed-point multiplications, thereby maximizing the area efficiency of the fused quantizer. In addition, the PoT term combinations for approximation are precomputed offline on a per-layer basis and statically mapped to the hardware to eliminate any additional runtime overhead.

4 Proposed hardware solution in AUTO-Accel and integrated hardware architecture

4.1 Adaptive unified buffer mapping strategy

Previous studies on MobileNetV1-based accelerators have adopted tiling techniques and double buffering structures to efficiently utilize OCM when processing large feature maps [22, 28, 31, 42]. In addition, in most of these studies, fixed-size memory allocations have been employed separately for activations and weights to handle parameter-specific memory requirements. However, as discussed in Sect. 2.1.2, MobileNetV1 exhibits opposing memory requirements for weights and activations depending on the network depth. This trend leads to persistently low utilization of each buffer allocated with a fixed size or causes frequent off-chip memory accesses.

Accordingly, this paper proposes AUBM, which adaptively allocates a fixed-size buffer according to the layer-wise memory requirement trends of MobileNetV1. AUBM is a technique that maintains high OCM utilization across CNN layers by storing the element with excessive memory demand among the two primary elements (*i.e.*, weights and activations) in the adaptive unified buffer (AUB). Figure 5 illustrates the reallocation of the AUB according to the layer characteristics of the network under the proposed AUBM technique. Specifically, Fig. 5a shows the reallocation of the AUB in the high-resolution region at the beginning of the network, where the AUB is allocated to store activations, efficiently addressing the high memory demands of the early layers with large feature maps. In addition, as shown in Fig. 5b, in the low-resolution region in the latter part of the network, the AUB is allocated for storing PWC weights to alleviate the heavy memory load associated with weights. In summary, by employing AUBM, which can flexibly adapt to the differing memory requirements of CNN weights and activations, it becomes

Fig. 4 Hardware architecture of the proposed SPoT-SA-based quantizer module

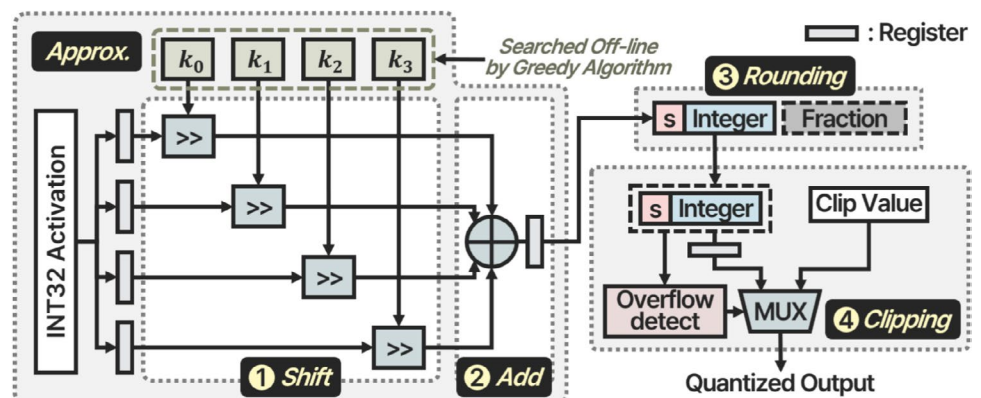


Fig. 5 Overview of the proposed AUBM. **a** Activation storage in early layers, **b** weight storage in later layers

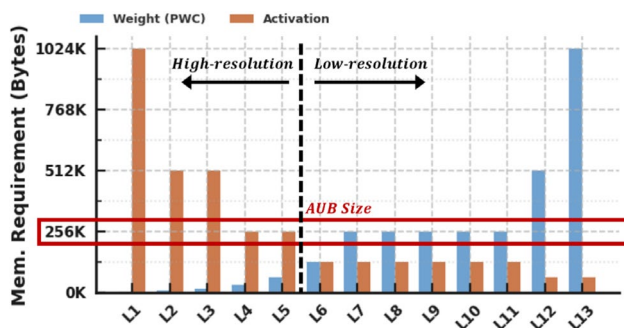
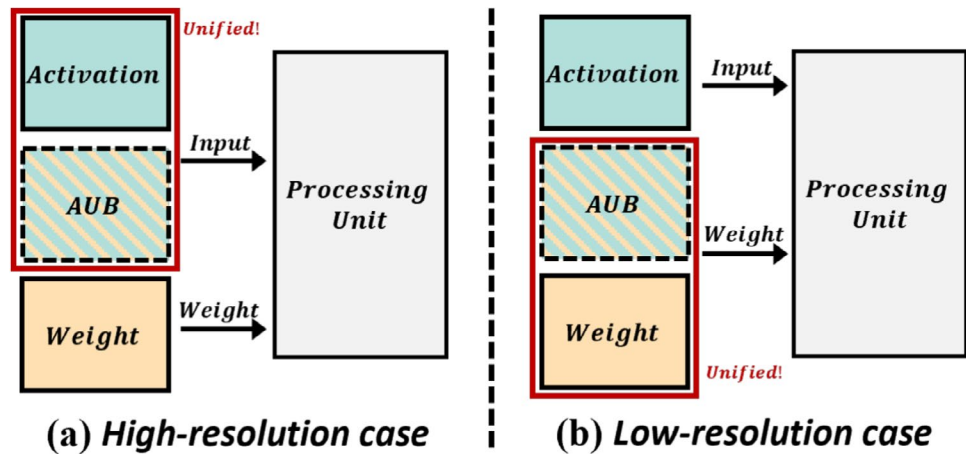


Fig. 6 Reduced memory footprint of weights and activations across MobileNetV1 layers after resizer

possible to effectively eliminate unnecessary memory usage caused by static memory allocation, while reducing OCM requirements.

Nevertheless, a large AUB size may lead to resource wastage in certain layers, while the opposite case can cause frequent off-chip memory accesses. Thus, selecting an appropriate AUB size requires balancing OCM usage with performance degradation due to off-chip access. To this end, layer-wise memory requirements for activations and weights in MobileNetV1-SSD are analyzed. Prior to this, using the resizer technique from Sect. 2.2.1, input activation memory was reduced by 27.18%. As shown in Fig. 6, the memory requirements for activations and weights become more uniform, often recurring at specific values (*i.e.*, 128 KB, 256 KB) in the middle layer intervals. Based on this, setting the AUB size to 128 KB results in more than twice the off-chip accesses compared to 256 KB. Therefore, the proposed accelerator adopts a 256 KB AUB size, which offers an optimal trade-off between memory efficiency and resource usage. With this configuration, layers before layer 6 are treated as high resolution, and the rest as low resolution, allowing better alignment with per-layer memory demands. The proposed AUBM and selected AUB size enable efficient

handling of asymmetric memory usage in CNNs while maintaining high OCM utilization.

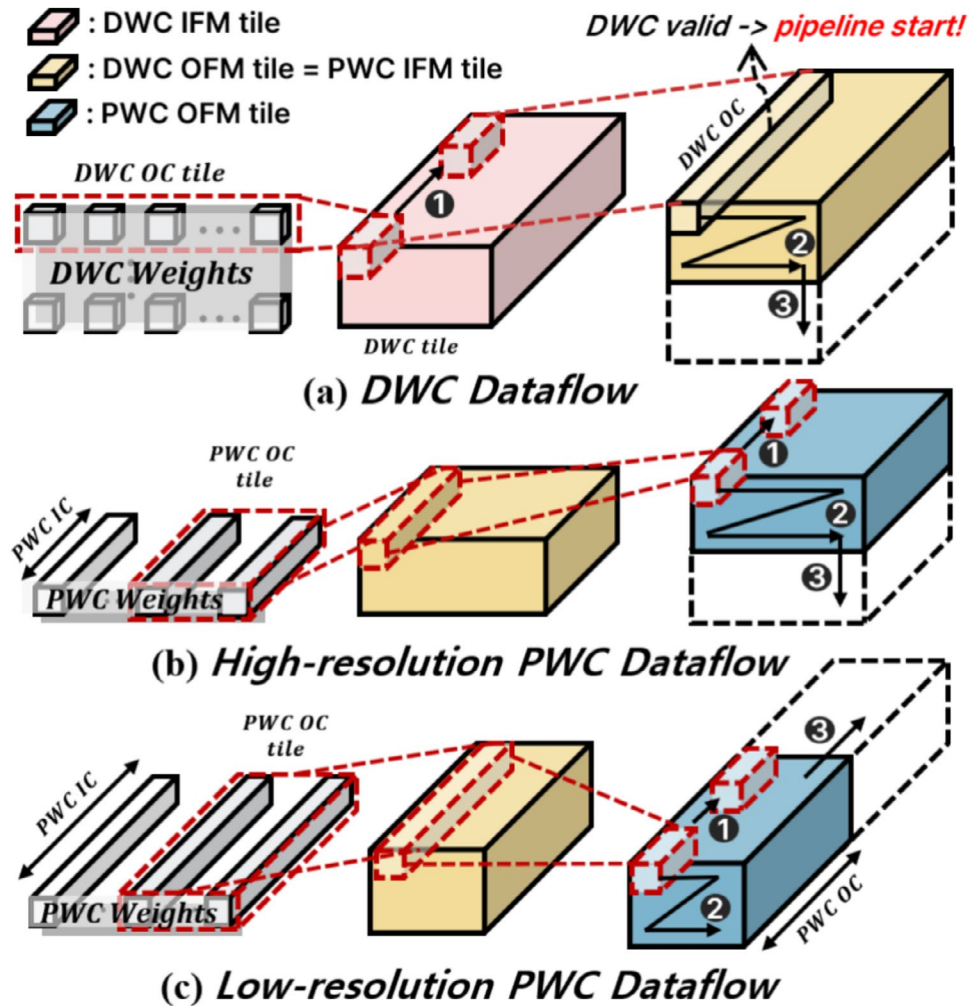
4.2 Tile-based adaptive pipelined dataflow

In CNNs, not only the memory requirements but also the data reuse rates vary significantly depending on the network depth. In such cases, applying a uniform dataflow across the entire network may lead to performance degradation in most layers except for a few. For example, in layers with high activation memory requirements, the limited capacity of the OCM necessitates storing data sequentially in tiles. Without employing an appropriate dataflow, this leads to an inefficient process in which the activation tiles previously stored in off-chip memory must be re-fetched during PWC operations. To address this issue, *AUTO-Accel* proposes TAP, a technique that dynamically adjusts the PWC dataflow according to the depth of CNN layers. TAP is designed to flexibly handle operations while avoiding unnecessary off-chip memory accesses in each region, based on the high-resolution and low-resolution regions defined by AUBM. Figure 7 illustrates the dataflow of TAP in the cases of high-resolution and low-resolution scenarios.

Figure 7a illustrates the computation dataflow for the DWC stage, which is identical for both high- and low-resolution cases. The input activations are stored in the OCM in units of DWC input feature map (IFM) tile, and the DWC outputs, referred to as DWC output feature map (OFM) tile, are generated sequentially across rows and columns within the DWC OC. These DWC OFM tiles are then pipelined into the PWC stage as input activations.

Figure 7b illustrates the computation dataflow in the PWC stage under the high-resolution case. Due to the large activation memory footprint, the design maximizes the reuse of activation tiles already loaded into the OCM. Specifically, PWC computations are performed along the PWC OC direction, expanding over rows and columns, with the output data

Fig. 7 Our proposed TAP dataflow. **a** Computation dataflow of DWC, **b** pipelined dataflow of high-resolution PWC dataflow, **c** pipelined dataflow of low-resolution PWC dataflow



from a single DWC OC being immediately pipelined into the PWC computations as soon as it is generated. This pipelined dataflow structure between the DWC and PWC stages allows all computations required for the generated DWC OFM tiles to be completed while it remains in the OCM, before proceeding to the computation of the next DWC IFM tile, thus completing the layer computation without requiring the data to be reloaded. Consequently, high-resolution OFM tiles are produced without re-fetching activation data, significantly reducing off-chip memory accesses and maximizing data reuse within the OCM.

In the low-resolution case, all activations fit in the OCM, but the large size of PWC weights limits them to be stored by PWC OC tiles. While the DWC stage follows the same dataflow as in the high-resolution case, applying the same to the PWC stage would require frequent re-fetching of PWC OC tile weights from off-chip memory. To prevent this, the proposed TAP technique uses a low-resolution-specific dataflow, shown in Fig. 7c. As each DWC OC output is generated, it is immediately pipelined into the PWC

stage. PWC computations proceed sequentially along rows, columns, and PWC OCs, ensuring that all operations using the loaded PWC OC tile weights are completed while they remain in the OCM. This approach enables efficient reuse of PWC weights and generates low-resolution OFM tile without repeated memory access.

The proposed TAP technique is organically integrated with the dynamic buffer allocation strategy of AUBM, providing a data reuse-centric dataflow based on the layer-wise memory requirement characteristics while maximizing OCM utilization. Empirically, the TAP technique demonstrates a 25% reduction in off-chip memory accesses compared to the case where the low-resolution dataflow is uniformly applied across all layers. This supports that the adaptive dataflow of TAP practically contributes to memory access optimization. Therefore, the proposed TAP not only optimizes data reuse but also enhances the efficiency of the AUB memory structure, thereby contributing to a tangible reduction in the cost associated with off-chip memory accesses at the overall system level.

4.3 MobileNetV1-SSD lite accelerator architecture

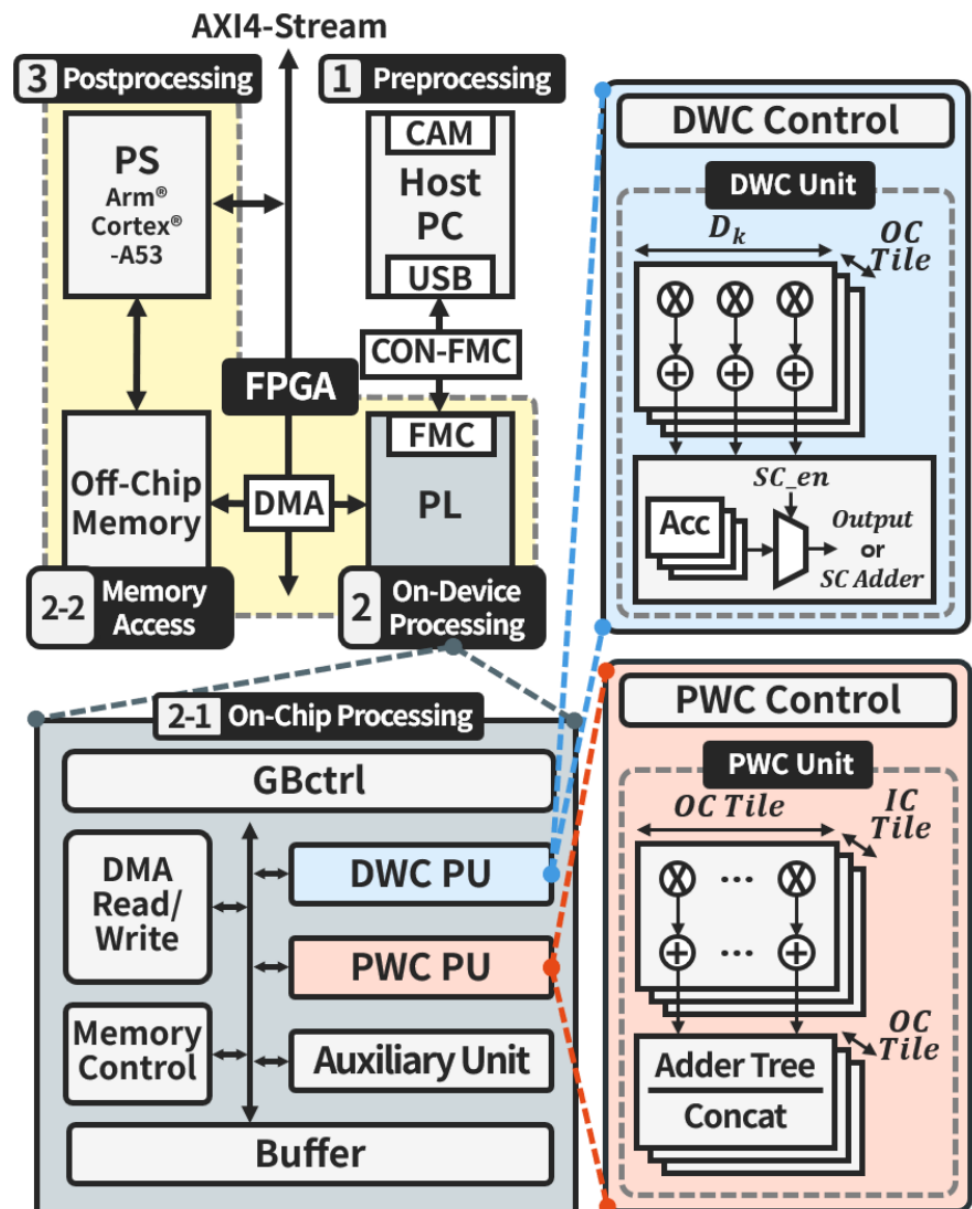
Figure 8 illustrates the overall architecture of the proposed accelerator, which is designed to efficiently process a MobileNetV1-SSD lite-based object detection model. The proposed accelerator primarily consists of a global controller (GBctrl), direct memory access (DMA) read/write modules, buffers, a processing unit (PU), and a PS.

The GBctrl manages the overall operation of the accelerator, coordinating the computation schedule based on information such as the feature map size and the number of channels for each layer. Through this, it controls the data flow to ensure that each module operates efficiently according to the computation sequence. The DMA module is responsible for data transmission between the off-chip memory and the

OCM, handling the loading and fetching of data required for computation, such as feature maps, weights, and biases, from the off-chip memory. The proposed accelerator includes a 256 KB AUB and is equipped with buffers capable of storing various data required for computation. To enhance the computation pipeline efficiency between DWC and PWC, the compute units are separated into dedicated PUs specialized for DWC and PWC operations. Noting that all layers of MobileNetV1-SSD lite, except for the input image channel, are composed with the number of channels in multiples of 32, the proposed accelerator is designed to support parallel computation in units of 32 ICs.

The DWC PU in Fig. 8 executes parallel computation over 96 pixels, employing 32 DWC OC tiles and a D_K value of 3. Considering the computational similarity between

Fig. 8 Overall architecture and on-chip processing flow of the proposed AUTO-Accel-based CNN inference accelerator. The system includes pre-processing via the host PC, inference with on-chip processing on the FPGA PL, off-chip memory access via DMA, and post-processing on the FPGA PS



DWC and SC in that both perform kernel operations, an SC-dedicated adder module is additionally incorporated, enabling the DWC PU to handle SC computations as well. The PWC PU is also designed to support parallel computation in units of 32 ICs, with the PWC IC tile set to 32 and the PWC OC tile set to 8, thereby enabling parallel computation on 256 pixels in total. This PWC PU configuration ensures that the PWC computation latency is always greater than the DWC computation output latency across all layers, allowing the TAP-based DWC-PWC pipeline proposed in this paper to operate efficiently without stalling. In addition, the auxiliary PU performs ReLU and SPoT-based fused quantization operations. After completing computations for all layers of MobileNetV1-SSD lite, the post-processing stage for detection is executed within the FPGA's PS. In this stage, bounding boxes and confidence scores are generated through non-maximum suppression (NMS).

4.4 Overall system processing flow

As presented in Fig. 8, the proposed *AUTO-Accel*-based system is designed with the goal of enabling low-latency processing and real-time object detection in edge environments. The entire system is composed of (1) pre-processing, (2) on-device processing, (2-1) on-chip processing, (2-2) memory access, and (3) post-processing stages, with the computations distributed across each stage to maximize processing efficiency.

First, in the (1) pre-processing stage, input images are transmitted from the camera (CAM) to the host PC, where they are resized to a resolution suitable for inference using the Resizer Network. The transformed images are then transferred to the FPGA PL via CON-FMC. Next, in the (2) on-device processing stage, object detection inference is performed on the FPGA-based MobileNetV1-SSD lite accelerator. This stage is structured such that (2-1) on-chip processing and (2-2) memory access are interlinked and executed iteratively, where necessary data is fetched from off-chip memory via the DMA module and loaded into the OCM as needed during computation. In this process, the proposed AUBM and TAP techniques minimize off-chip memory accesses while maximizing OCM utilization and computational efficiency. In addition, SPoT-SA-based fused quantization reduces the area and power consumption of the compute units while minimizing accuracy degradation. Finally, in the (3) post-processing stage, the feature maps from the completed inference are processed within the FPGA's PS to generate bounding boxes and confidence scores through SSD post-processing, producing the final object detection results. Through this SW/HW co-design-based structure, the system enables the MobileNetV1-SSD lite-based object detection model to be processed in real time

with low latency and low power consumption in edge environments, realizing a system optimized for real-time object detection inference.

5 Experimental results

5.1 Experimental settings

To validate the proposed SW/HW solution, a MobileNetV1-based object detection model with an integrated resizer was trained on the Pascal VOC [8] dataset using PyTorch on an NVIDIA RTX 2080 Ti GPU, following the hyperparameters from prior works [14, 32]. The hardware performance was evaluated on an Xilinx ZCU102 evaluation board, and the register-transfer level (RTL) implementation was performed using Xilinx Vivado 2023.1. In addition, the power consumption of the proposed accelerator was measured using the Vivado power estimation tool.

5.2 Performance of HW solution

5.2.1 Comparison results with the previous studies

Tables 1 and 2 present the performance and HW resource usage comparisons between the proposed accelerator and existing FPGA accelerators for MobileNet and MobileNet-backbone SSD models, respectively. As the primary evaluation metric, throughput is expressed in giga operations per second (GOPS), while energy efficiency is represented as GOPS/W, calculated by dividing the throughput by the power consumption in watts. HW efficiency, defined as GOPS/DSP, measures throughput per DSP, the primary compute resource in the system. The tables also report the usage of major HW resources, including BRAM, DSP, LUT, and FF.

As shown in Table 1, the proposed accelerator achieves a throughput of 107.67 GOPS while consuming 5.5 W of power, utilizing only 53 DSPs, and demonstrating a BRAM utilization of 320 for the MobileNetV1 model. Notably, in terms of energy efficiency, the proposed accelerator achieves 19.57 GOPS/W, which is 7.52% higher than the best among previous studies [13], demonstrating superior performance compared to existing works [13, 24, 27, 38, 40] by a factor of 1.08× to 21.04×. Moreover, in terms of HW efficiency, the proposed accelerator achieves an impressive 2.03 GOPS/DSP, showing a 1.29× to 9.66× improvement compared to previous works [13, 38, 40].

As shown in the SSD-based accelerator comparison results in Table 2, the proposed accelerator achieves 11.502 FPS/W and 1.194 FPS/DSP, demonstrating overall superior performance compared to previous works [1, 9, 19, 21, 36, 37]. Specifically, compared to Ref. [9], which uses a

Table 1 Performance evaluation with existing MobileNetV1 accelerator architectures

	CAC'22 [27]	IAEAC'19 [24]	TCAS II'24 [13]	FPGA'20 [40]	TCAS I'21 [38]	Ours
Platform	XC7Z045	XC7Z045	XCZU9EG	XCK325T	Arria 10	XCZU9EG
Freq.(MHz)	150	100	150	200	200	200
BRAMs	537	110.5	506.5	193.5	769	320
DSPs	787	109	69	704	607	53
LUTs/FFs (k)	154/128	9.2/16.9	214.9/166.7	173.5/241.2	*207/–	112.7/68.3
Image resolution	224*224	224*224	224*224	224*224	–	256*256
Data format	Fixed-8	Fixed-16	INT-8	Fixed-8	Fixed-8	INT-8
Speed (FPS)	5.5	0.35	190.9	264.6	235.5	67.72
Throughput (GOPS)	13	2	108.3	147.84	267.08	107.67
Power (W)	6.24	2.15	5.95	8.26	–	5.5
Energy efficiency (GOPS/W)	2.08	0.93	18.2	17.9	–	19.57
HW efficiency (GOPS/DSP)	0.017	0.018	1.57	0.21	0.44	2.03

Bold values indicate better results compared to other accelerators

*Values marked with * represent adaptive logic module (ALM) usage instead of LUTs

Table 2 Performance evaluation with existing SSD accelerators based on MobileNet architectures

	FPL'19 [37]	FPL'21 [1]	ICCV'19 [21]	FPT'18 [9]	VLSI'21 [19]		AI'25 [36]	Ours
Model	MobileNetV1-SSD	MobileNetV1-SSD	Custom MobileNetV1-SSD	Custom MobileNetV2-SSD lite	MobileNetV2-SSD lite		Mobile-NetV1-SSD	MobileNetV1-SSD lite
Platform	XCZU9EG	Stratix 10 GX2800	XCZU3EG	XC7Z045	Stratix IV GX	Arria 10	XCVU13P	XCZU9EG
Freq.(MHz)	430	350	215	100	100	200	300	200
BRAMs	145	531	178.5	311	126	215	8808	320
DSPs	212	5129	83	728	624	980	4195	53
LUTs/FFs (k)	31.2/46.8	*575/–	50/74	148.5/191.9	*68/–	*94/–	898.6/–	112.7/68.3
Dataset	COCO	COCO	VOC	VOC	VOC+COCO	VOC+COCO	VOC	VOC
Image resolution	300*300	300*300	512*512	224*224	300*300	300*300	300*300	256*256
Data format	Fixed-8	Fixed-16	Mixed	Mixed	Fixed-16	Fixed-16	Fixed-16	INT-8
Task Acc.(%)	19.3	22.5	66.4	20.3	78.8	78.8	–	55.78
Speed (FPS)	31	609	18	64.8	24.9	84.8	152.41	63.26
Throughput (GOPS)	49.29	–	46.8	–	–	–	–	100.58
Power (W)	–	–	6.9	9.9	5.1	9.8	62.1	5.5
Energy efficiency (FPS/W)	–	–	2.609	6.545	4.882	8.653	2.454	11.502
HW efficiency (FPS/DSP)	0.146	0.119	0.217	0.089	0.040	0.087	0.036	1.194

*Values marked with * represent ALM usage instead of LUTs

similar amount of BRAM resources, the proposed accelerator achieves a 13.42× improvement in HW efficiency while using 13.74× fewer DSPs, with only a 2.38% degradation in processing speed. These results can also be interpreted as the outcome of the proposed AUBM technique efficiently allocating OCM in accordance with the layer-wise memory

requirements, while the TAP technique maximizes data reuse, effectively reducing off-chip memory accesses. In other words, in accelerator implementations where tiling is applied, the organically integrated design of flexible buffer mapping that reflects the memory requirements between activations and weights, along with dynamic computation

flow, is critical. The proposed HW solution demonstrates that it can practically improve not only computational performance but also power and resource efficiency.

5.2.2 Effectiveness analysis of each proposed method

Table 3 presents the changes in performance and resource utilization according to the application of each proposed technique. The naive accelerator without any of the proposed techniques applied exhibits a low computational performance of 47.21 GOPS while consuming 845 BRAMs. By introducing the resizer to reduce the input resolution, the throughput was improved by approximately 27.2%, and the BRAM usage was reduced to 761, alleviating the memory resource burden. Applying the TAP technique along with the resizer enabled hiding the computation latency of the DWC through the DWC-PWC computation pipeline, achieving approximately a 2.13× improvement in throughput while consuming only an additional 39.4k LUTs and 25.6k FFs. Furthermore, with the application of the AUBM technique, the BRAM usage was reduced to 320, indicating a 57.94% decrease. This clearly demonstrates the effectiveness of AUBM in enhancing the efficiency of OCM utilization. Finally, by introducing the multiplier-free SPoT-SA-based fused quantizer in place of the conventional fused quantizer module, a 10.36× reduction in DSP resource usage was achieved. This suggests that the SPoT-SA-based fused quantizer module can significantly contribute to HW resource savings while maintaining computational performance. As a result, each proposed technique not only provides meaningful performance improvements independently but also works complementarily within the overall accelerator system, clearly demonstrating the ability to enhance practical processing performance and HW resource efficiency simultaneously.

In addition, to provide an intuitive evaluation of the memory efficiency of the proposed accelerator, we applied the roofline model analysis. The roofline model configuration follows Ref. [41], where the x -axis represents the computation to communication (CTC) ratio and the y -axis denotes throughput. The CTC ratio is defined as the total number of operations divided by the number of DRAM

accesses. Figure 9 illustrates the roofline analysis of the proposed accelerator. Specifically, P1 corresponds to the naive version without the resizer and the proposed optimization techniques, P2 represents the version with the resizer applied, and P3 shows the version with both the resizer and the TAP applied. In P1, the absence of optimization leads to excessive external memory accesses, resulting in a low CTC ratio and reduced throughput. In P2, introducing the resizer reduces the overall number of operations and shrinks the global feature map size, enabling better alignment with the optimal AUBM configuration. Consequently, DRAM accesses decrease and throughput improves slightly. However, since the reduction in operations outweighs the reduction in DRAM accesses, the CTC ratio becomes lower than that of P1. As a result, data reuse remains limited, preventing throughput from being maximized. Finally, in P3, applying TAP in addition to the resizer significantly increases data reuse, which substantially reduces DRAM accesses. At the same time, the utilization of compute resources is maintained at a high level across layers, leading to a considerable improvement in throughput that approaches the roofline performance bound. This analysis demonstrates that the proposed optimization techniques not only provide

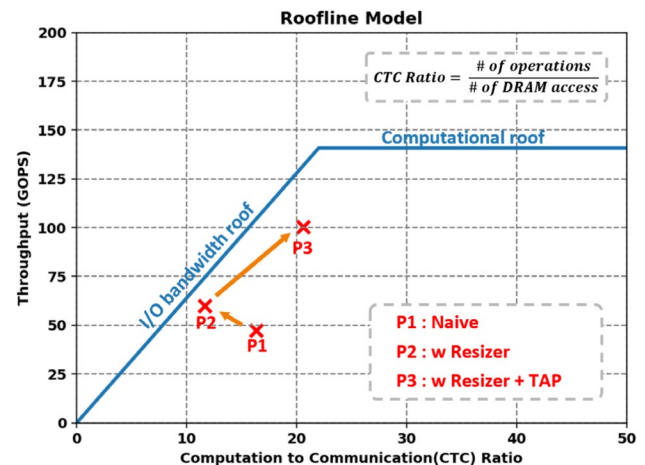


Fig. 9 Roofline model analysis of the proposed accelerator across naive, resizer, and TAP optimizations

Table 3 Performance change according to each method

	FPS	Throughput	BRAMs	DSPs	LUTs (k)	FFs (k)
Naive	20.77	47.21	845	549	44.8	33.1
+R	38.14	60.04	761	549	44.8	33.1
+R, T	63.26	100.58	761	549	84.2	58.7
+R, T, A	63.26	100.58	320	549	84.2	58.7
+R, T, A, S	63.26	100.58	320	53	112.7	68.3

Bold values indicate performance improvements achieved by applying each method

*R: resizer, A: AUBM, T: TAP, S: SPoT-SA

Table 4 Comparison of detection accuracy on the Pascal VOC dataset

Resolution	W/A	Resizer	Approx.	mAP(%)
256 × 256	32/32	X	X	53.25
256 × 256	32/32	O	X	59.57
256 × 256	8/8	O	X	58.56
256 × 256	8/8	O	PoT	28.23
256 × 256	8/8	O	SPoT-SA	58.63

individual benefits but also jointly improve both memory access efficiency and computational performance.

5.3 Ablation studies for SW solutions

To develop an efficient object detection model, this subsection evaluates two SW optimization methods: input resolution adjustment and precise quantization techniques. Table 4 presents the mean average precision (mAP) values for each resizer and quantization method. Reducing the resolution significantly lowers the computational load but inevitably leads to a drop in mAP. To mitigate this, the proposed resizer improves mAP from 53.25% to 59.42%, confirming that the 256 × 256 configuration with the resizer provides a balance between performance and computational efficiency. Next, to evaluate the SPoT-SA-based fused quantization, three methods were compared: W8/A8 uniform quantization, single PoT-based scaling factor approximation, and the SPoT-SA method. Under the W8/A8 (i.e., weight 8-bit, activation 8-bit) quantization environment, the PoT-based method exhibited a severe mAP degradation of − 25.02% compared to the baseline (i.e., 256 × 256 (w/o resizer)). In contrast, the SPoT-SA method, through more precise scaling factor approximation, demonstrated that it can minimize accuracy degradation (5.38% higher than W32/A32 w/o resizer & 0.94% lower than W32/A32 w/ resizer) while maintaining an HW-friendly solution.

6 Conclusion

In this paper, we proposed *AUTO-Accel*, a SW/HW co-design optimization solution for the efficient deployment of the CNN-based object detection model, MobileNetV1-SSD lite, in edge environments. On the SW side, we simultaneously applied the learnable resizer network, a previously proposed image pre-processing technique, and the proposed HW-friendly quantization technique, SPoT-SA-based fused quantization, to reduce memory and HW resource usage while minimizing accuracy degradation. On the HW side, AUBM and TAP were introduced to maximize computational pipeline efficiency by increasing OCM utilization and minimizing off-chip memory accesses. The proposed

accelerator achieved an energy efficiency of 11.5 FPS/W, demonstrating a 1.33× to 4.41× improvement over existing MobileNet-based FPGA accelerators. In conclusion, *AUTO-Accel* presents an integrated optimization solution considering both SW and HW in the HW deployment of object detection models, achieving higher energy efficiency compared to previously proposed accelerators.

Acknowledgements This research was partly supported by K-CHIPS (Korea Collaborative & Hightech Initiative for Prospective Semiconductor Research) (RS-2025-02305531) funded by the Ministry of Trade, Industry & Energy (MOTIE, Korea) and the MSIT (Ministry of Science and ICT), Korea, under the ITRC (Information Technology Research Center) support program (IITP-2025-RS-2022-00156295) supervised by the IITP (Institute for Information & Communications Technology Planning & Evaluation). The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

Author contributions In the research project, JK and DL equally contributed to conceptualization, drafting, data curation, analysis, visualization, and validation. YK was involved in data curation and draft preparation, with YK and DL also contributing to visualization. HK oversaw the project, managed administration, secured funding, provided resources, and reviewed and edited the manuscript.

Data availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Anupreetham, A., Ibrahim, M., Hall, M., Boutros, A., Kuzhiveli, A., Mohanty, A., Nurvitadhi, E., Betz, V., Cao, Y., Seo, J.S.: End-to-end FPGA-based object detection using pipelined CNN and non-maximum suppression. In: 2021 31st International Conference on Field-Programmable Logic and Applications (FPL), pp. 76–82. IEEE (2021)
2. Bae, S.H., Lee, H.J., Kim, H.: MCM-SR: multiple constant multiplication-based CNN streaming hardware architecture for super-resolution. In: IEEE Transactions on Very Large Scale Integration (VLSI) Systems (2024)
3. Chen, Y., Dai, X., Chen, D., Liu, M., Dong, X., Yuan, L., Liu, Z.: Mobile-former: bridging mobilenet and transformer. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 5270–5279 (2022)
4. Choi, D., Kim, H.: GradQ-VIT: Robust and efficient gradient quantization for vision transformers. Proc. AAAI Confer. Artif. Intell. **39**(15), 16019–16027 (2025). <https://doi.org/10.1609/aaai.v39i15.33759>
5. Choi, D., Park, J., Kim, H.: HLQ: hardware-friendly logarithmic quantization aware training for power-efficient low-precision CNN models. IEEE Access. **12**, 159611–159621 (2024)
6. Choi, J., Chun, D., Kim, H., Lee, H.J.: Gaussian yolov3: an accurate and fast object detector using localization uncertainty for autonomous driving. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 502–511 (2019)
7. Colbert, I., Pappalardo, A., Petri-Koenig, J.: A2Q: accumulator-aware quantization with guaranteed overflow avoidance. In:

- Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 16989–16998 (2023)
8. Everingham, M., Van Gool, L., Williams, C.K., Winn, J., Zisserman, A.: The pascal visual object classes (VOC) challenge. *Int. J. Comput. Vis.* **88**, 303–338 (2010)
 9. Fan, H., Liu, S., Ferianc, M., Ng, H.C., Que, Z., Liu, S., Niu, X., Luk, W.: A real-time object detection accelerator with compressed SSDLite on FPGA. In: 2018 International Conference on Field-programmable Technology (FPT), pp. 14–21. IEEE (2018)
 10. He, K., Gkioxari, G., Dollár, P., Girshick, R.: Mask R-CNN. In: Proceedings of the IEEE International Conference on Computer Vision (ICCV) (2017)
 11. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2016)
 12. Hettiarachchi, D.L.N., Davuluru, V.S.P., Balster, E.J.: Integer vs. floating-point processing on modern FPGA technology. In: 2020 10th Annual Computing and Communication Workshop and Conference (CCWC), pp. 0606–0612. IEEE (2020)
 13. Hong, H., Choi, D., Kim, N., Kim, H.: Mobile-X: dedicated FPGA implementation of the mobilenet accelerator optimizing depthwise separable convolution. In: IEEE Transactions on Circuits and Systems II: Express Briefs (2024)
 14. Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H.: Mobilenets: efficient convolutional neural networks for mobile vision applications. Preprint at [arXiv:1704.04861](https://arxiv.org/abs/1704.04861) (2017)
 15. Huang, J., Rathod, V., Sun, C., Zhu, M., Korattikara, A., Fathi, A., Fischer, I., Wojna, Z., Song, Y., Guadarrama, S., et al.: Speed/accuracy trade-offs for modern convolutional object detectors. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 7310–7311 (2017)
 16. Jeong, S., Lee, S., Kim, H.: LowgradQ: adaptive gradient quantization for low-bit CNN training via kernel density estimation-guided thresholding and hardware-efficient stochastic rounding unit. In: 2025 Design, Automation & Test in Europe Conference (DATE), pp. 1–2. IEEE (2025)
 17. Kim, N.J., Kim, H.: FP-AGL: filter pruning with adaptive gradient learning for accelerating deep convolutional neural networks. *IEEE Trans. Multimed.* **25**, 5279–5290 (2022)
 18. Kim, N.J., Lee, J., Kim, H.: HYQ: hardware-friendly post-training quantization for CNN-transformer hybrid networks. In: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, pp. 4291–4299 (2024)
 19. Kim, S., Na, S., Kong, B.Y., Choi, J., Park, I.C.: Real-time SSDLite object detection on FPGA. *IEEE Trans. Very Large Scale Integr. VLSI Syst* **29**(6), 1192–1205 (2021)
 20. Koo, K., Kim, H.: V-SKP: vectorized kernel-based structured kernel pruning for accelerating deep convolutional neural networks. *IEEE Access* **11**, 118547–118557 (2023)
 21. Li, F., Mo, Z., Wang, P., Liu, Z., Zhang, J., Li, G., Hu, Q., He, X., Leng, C., Zhang, Y., et al.: A system-level solution for low-power object detection. In: Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (2019)
 22. Li, G., Liu, Z., Li, F., Cheng, J.: Block convolution: toward memory-efficient inference of large-scale CNNs on FPGA. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* **41**(5), 1436–1447 (2021)
 23. Li, Y., Dong, X., Wang, W.: Additive powers-of-two quantization: an efficient non-uniform discretization for neural networks. Preprint at [arXiv:1909.13144](https://arxiv.org/abs/1909.13144) (2019)
 24. Liao, J., Cai, L., Xu, Y., He, M.: Design of accelerator for mobilenet convolutional neural network based on FPGA. In: 2019 IEEE 4th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC), vol. 1, pp. 1392–1396. IEEE (2019)
 25. Liu, S., Deng, W.: Very deep convolutional neural network based image classification using small training sample size. In: 2015 3rd IAPR Asian Conference on Pattern Recognition (ACPR), pp. 730–734 (2015). <https://doi.org/10.1109/ACPR.2015.7486599>
 26. Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.Y., Berg, A.C.: SSD: single shot multibox detector. In: Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14, pp. 21–37. Springer (2016)
 27. Liu, W., Li, Y., Yang, Y., Zhu, J., Liu, L.: Design an efficient DNN inference framework with PS-PL synergies in FPGA for edge computing. In: 2022 China Automation Congress (CAC), pp. 4186–4190. IEEE (2022)
 28. Ma, Y., Cao, Y., Vrudhula, S., Seo, J.S.: Optimizing loop operation and dataflow in FPGA acceleration of deep convolutional neural networks. In: Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, pp. 45–54 (2017)
 29. Mehta, S., Rastegari, M.: Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer. In: International Conference on Learning Representations (2022). <https://openreview.net/forum?id=vh-0sUt8HIG>
 30. Peng, P., You, M., Xu, W., Li, J.: Fully integer-based quantization for mobile convolutional neural network inference. *Neurocomputing* **432**, 194–205 (2021)
 31. Podili, A., Zhang, C., Prasanna, V.: Fast and efficient implementation of convolutional neural networks on FPGA. In: 2017 IEEE 28th International Conference on Application-specific Systems, Architectures and Processors (ASAP), pp. 11–18. IEEE (2017)
 32. Talebi, H., Milanfar, P.: Learning to resize images for computer vision tasks. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 497–506 (2021)
 33. Tan, M., Le, Q.: Efficientnet: Rethinking model scaling for convolutional neural networks. In: International Conference on Machine Learning, pp. 6105–6114. PMLR (2019)
 34. Tan, M., Pang, R., Le, Q.V.: Efficientdet: scalable and efficient object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 10781–10790 (2020)
 35. Tu, Z., Milanfar, P., Talebi, H.: Muller: multilayer Laplacian resizer for vision. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 6877–6887 (2023)
 36. Wan, Y., Chen, J., Yang, X., Zhang, H., Huang, C., Xie, X.: DSA-CNN: an FPGA-integrated deformable systolic array for convolutional neural network acceleration. *Appl. Intell.* **55**(1), 65 (2025)
 37. Wu, D., Zhang, Y., Jia, X., Tian, L., Li, T., Sui, L., Xie, D., Shan, Y.: A high-performance CNN processor based on FPGA for mobilenets. In: 2019 29th International Conference on Field Programmable Logic and Applications (FPL), pp. 136–143. IEEE (2019)
 38. Wu, X., Ma, Y., Wang, M., Wang, Z.: A flexible and efficient FPGA accelerator for various large-scale and lightweight CNNs. *IEEE Trans. Circuits Syst. I Regul. Pap.* **69**(3), 1185–1198 (2021)
 39. Xiong, Y., Liu, H., Gupta, S., Akin, B., Bender, G., Wang, Y., Kindermans, P.J., Tan, M., Singh, V., Chen, B.: MobileDets: searching for object detection architectures for mobile accelerators. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 3825–3834 (2021)
 40. Yu, Y., Zhao, T., Wang, K., He, L.: Light-OPU: an FPGA-based overlay processor for lightweight convolutional neural networks. In: Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, pp. 122–132 (2020)
 41. Zhang, C., Li, P., Sun, G., Guan, Y., Xiao, B., Cong, J.: Optimizing FPGA-based accelerator design for deep convolutional neural networks. In: Proceedings of the 2015 ACM/SIGDA International

- Symposium on Field-programmable Gate Arrays, pp. 161–170 (2015)
42. Zhang, C., Sun, G., Fang, Z., Zhou, P., Pan, P., Cong, J.: Caf-feine: toward uniformed representation and acceleration for deep convolutional neural networks. *IEEE Trans. Comput. Aided Des. Integr. Circ. Syst.* **38**(11), 2072–2085 (2018)
43. Zheng, Y., Yang, H., Shu, Y., Jia, Y., Huang, Z.: Optimizing off-chip memory access for deep neural network accelerator. *IEEE Trans. Circ. Syst. II Express Briefs* **69**(4), 2316–2320 (2022)
44. Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., Ling, H.: Detection and tracking meet drones challenge. *IEEE Trans. Pattern Anal. Mach. Intell.* **44**(11), 7380–7399 (2021)
45. Zou, Z., Chen, K., Shi, Z., Guo, Y., Ye, J.: Object detection in 20 years: a survey. *Proc. IEEE* **111**(3), 257–276 (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.